

Bom dia chat! Pode me ajudar com uma coisa? Preciso que você me de um baita resumao sobre Lisp, mas a parte pratica mesmo, sintaxes, e monta um codigo simples pra eu entender

Bom dia, Caio!

Se a ideia é entender **Lisp para usar na prática**, sem ficar preso na história da linguagem, vou te passar um resumo focado em sintaxe e exemplos.

O que é Lisp?

Lisp é uma das linguagens de programação mais antigas que ainda existem.

A principal característica dela é:

Tudo é uma lista.

Enquanto em Java você escreve:

```
Java
```

Em Lisp você escreve:

```
lisp
```

Perceba que o operador vem primeiro.

Isso é chamado de **notação prefixa**.

Estrutura Básica

Tudo fica entre parênteses:

```
lisp
```

Exemplos:

```
lisp
```

Resultado:

```
lisp
```

Multiplicação:

```
lisp
```

Resultado:

```
lisp
```

Divisão:

```
lisp
```

Resultado:

```
lisp
```

Operações Aninhadas

Em Java:

Java

Em Lisp:

`lisp`

Primeiro:

`lisp`

vira:

`lisp`

Depois:

`lisp`

Resultado:

`lisp`

Variáveis

Criar uma variável:

`lisp`

ou

`lisp`

Consultar:

`lisp`

Resultado:

`lisp`

Strings

`lisp`

Mostrar:

`lisp`

Saída:

`lisp`

Listas

A coisa mais importante do Lisp.

Criando:

`lisp`

```
(setq numeros '(1 2 3 4 5))
```

ou

```
lisp
```

Acessar primeiro elemento:

```
lisp
```

Resultado:

```
lisp
```

Restante da lista:

```
lisp
```

Resultado:

```
lisp
```

Condicionais (IF)

Java:

```
Java
```

Lisp:

```
(if (= idade 10)
```

lisp

Exemplo:

```
(setq idade 21)
```

lisp

```
(print "nao pode atingir "))
```

Resultado:

lisp

Operadores de Comparação

Maior:

lisp

Menor:

lisp

Maior ou igual:

lisp

Menor ou igual:

lisp

Igual:

lisp

Funções

Java:

```
public static int soma(int a, int b)  
{
```

Java

Lisp:

lisp

Usando:

lisp

Resultado:

lisp

Laços

Loop simples

```
lisp
```

Saída:

```
0
```

```
1
```

```
lisp
```

```
→
```

Percorrer lista

```
(setq carros '(Lancer Civic Jetta Golf))
```

```
lisp
```

```
;;
```

Saída:

```
Lancer
```

```
lisp
```

```
--
```

Exemplo Completo

Vamos fazer algo parecido com os exercícios que você vê na faculdade.

Sistema simples de aluno:

```
(defun verificar-nota (nota)
```

```
(print "Nota do aluno:")
```



```
(print nota)

(if (>= nota 60)

  (print "Aprovado")

  lisp
)

(verificar-nota 75)
```

Saída:

```
Nota do aluno:
lisp
```

Exemplo mais próximo de Paradigma Lógico

Você comentou que está vendo Prolog e paradigmas de programação.

Em Lisp é comum trabalhar recursivamente.

Somar todos os números de uma lista:

```
(defun soma-lista (lista)

  (if (null lista)

      0

      lisp
      ,
  )
)
```

Usando:

```
lisp
```

Resultado:

lisp

Tradução mental Java → Lisp

Java	Lisp
variável	setq
método	defun
if	if
for	dotimes
ArrayList	lista
return	último valor da função
{ }	()
soma(5,3)	(soma 5 3)
5 + 3	(+ 5 3)

O que costuma cair em prova de Lisp?

Normalmente:

1. Entender a notação prefixa

lisp

2. Criar funções com defun
3. Trabalhar com listas

car

lisp

```
list
```

4. Fazer condicionais

```
lisp
```

5. Fazer recursão

```
defun
```

```
lisp
```

Se você conseguir ler e escrever funções usando `defun`, `if`, `car`, `cdr` e recursão, já entende uns 80% do Lisp que normalmente aparece em disciplinas de paradigmas de programação.



como faço pra pessoa entrar com um valor? inserir uma variavel?

Em Lisp, para o usuário digitar um valor, normalmente você usa a função `read`.

Exemplo simples:

```
lisp
```

Se o usuário digitar:

```
21
```

A variável `idade` receberá:

```
lisp
```

21

Um exemplo mais completo:

```
(print "Digite sua idade:")
```

lisp

```
(print "Digite sua idade:")
```

Saída:

```
"Digite sua idade:"
```

```
21
```

```
21
```

Com texto (string)

Se quiser que o usuário digite um nome:

```
(print "Digite seu nome:")
```

lisp

O usuário deve digitar:

```
"Caio"
```

(com aspas)

Exemplo de programa completo

```
(print "Digite o primeiro numero:")
```

```
(setq n1 (read))
```

```
(print "Digite o segundo numero:")
```

```
(setq n2 (read))
```

lisp

```
(print resultado. )  
(print soma)
```

Entrada:

```
10  
20
```

Saída:

```
"Resultado:"  
30
```

Criando uma função

```
(defun calcular-media ()
```

```
  (print "Nota 1:")  
  (setq n1 (read))
```

```
  (print "Nota 2:")
```

lisp

```
  (/ (+ n1 n2) 2)  
)
```

Chamando:

lisp

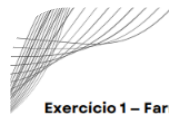
Se digitar:

```
80  
60
```

Resultado:

70

Uma observação importante: dependendo do interpretador Lisp que seu professor usa (Common Lisp, Scheme, Racket, CLISP, SBCL etc.), o comando de entrada pode mudar um pouco. Se você me falar **qual Lisp está usando na matéria** (por exemplo, Common Lisp, CLISP, Racket/Scheme), eu posso te mostrar exatamente a sintaxe correta para a sua aula.

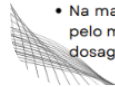


EXERCÍCIO 1

Exercício 1 – Farmácia da Maomao

Maomao precisa calcular a dosagem e o preço das ervas que prepara para seus pacientes. Cada erva tem um preço base, e a dosagem varia de acordo com o peso e a idade de quem vai tomar.

- Defina um tipo `Erva` com os campos `nome` e `precoBase`.
- Implemente `calculaDosagem :: Double -> Int -> String` que recebe peso e idade e usa `guards` para retornar a dosagem como texto – "10ml" para crianças pequenas ou abaixo do peso, "25ml" para a faixa intermediária e "50ml" para os demais casos.
- Implemente `ajustaPreco :: Erva -> Double` que usa `pattern matching` no nome da erva para aplicar o fator de preço – "Ginseng" multiplica por 3.0, "Lotus" multiplica por 1.5 e qualquer outro retorna o preço base sem alteração.
- Implemente `descricaoErva :: Erva -> Double -> Int -> String` que recebe uma erva, o peso e a idade do paciente e retorna uma `String` com o nome da erva, o preço ajustado e a dosagem indicada.
- Na `main`, crie três ervas – "Ginseng", "Lotus" e "Camomila" – e exiba a descrição para pelo menos dois pacientes com idades e pesos diferentes, cobrindo todos os casos de dosagem.



09

vou te mostrar o exercicio q eu preciso fazer, nao precisa montar o codigo, so separa por etapas e me explica como fazer cada etapa

Claro. Esse exercício é em **Haskell**, não Lisp.

Você pode separar em **5 etapas**:

1. Criar o tipo `Erva`

Você precisa criar uma estrutura com dois campos:

`haskell`

`nome`

`precoBase`

A ideia é representar uma erva, por exemplo:

Ginseng com preço base 10.0

Lotus com preço base 8.0

Camomila com preço base 5.0

2. Criar a função `calculaDosagem`

Ela recebe:

```
haskell
```

```
peso idade
```

E devolve um texto:

```
haskell
```

```
String
```

A lógica é:

Se for criança pequena OU abaixo do peso:

```
"10ml"
```

Se for adulto:

```
"25ml"
```

Para os demais casos:

```
"50ml"
```

Aqui você vai usar `guards` , tipo:

```
haskell
```

```
| condição = resultado
```

```
| condição = resultado
```

```
| otherwise = resultado
```

3. Criar a função `ajustaPreco`

Ela recebe uma `Erv` e retorna o preço ajustado.

Regras:

```
Ginseng -> preçoBase * 3.0
Lotus   -> preçoBase * 1.5
Qualquer outra -> preçoBase sem alteração
```

Aqui o exercício pede **pattern matching** no nome da erva.

A ideia é verificar o campo `nome` .

4. Criar `descricaoErva`

Essa função junta tudo.

Ela recebe:

```
haskell
```

```
erva peso idade
```

E retorna uma frase com:

```
nome da erva
preço ajustado
dosagem indicada
```

Exemplo de saída:

```
Erva: Ginseng | Preço: 30.0 | Dosagem: 25ml
```

Dentro dela você usa:

```
haskell
```

```
ajustaPreco erva
calculaDosagem peso idade
```

5. Montar o `main`

No `main` , você precisa:

1. Criar três ervas:

- Ginseng
- Lotus
- Camomila

2. Criar pelo menos dois pacientes com pesos e idades diferentes.
3. Exibir as descrições usando `putStrLn`.

O importante é testar todos os casos de dosagem:

```
10ml
25ml
50ml
```

Então escolha pacientes que façam cair nessas três situações.



pra criar o tipo é `defstruct` né?

Depende da linguagem que o professor está usando.

Pelo enunciado, como aparece:

haskell

```
Erva -> Double
Erva -> Double -> Int -> String
```

isso é **sintaxe de Haskell**, não Lisp.

Em **Haskell** você não usa `defstruct`.

Você criaria algo parecido com:

haskell

```
data Erva = Erva
  { nome :: String
  , precoBase :: Double
  }
```

ou até:

haskell

```
data Erva = Erva String Double
```

O `defstruct` é usado em algumas implementações de **Lisp** (como Common Lisp):

lisp

```
(defstruct erva
  nome
  precoBase)
```

Aí você criaria uma erva assim:

lisp

```
(make-erva :nome "Ginseng" :precoBase 10)
```

Então a primeira coisa é confirmar:

📌 Esse exercício é para **Haskell** ou para **Lisp**?

Pelo formato das assinaturas (`Erva -> Double -> Int -> String`), parece 99% Haskell. Se você me mostrar o modelo que o professor passou ou o início do arquivo, consigo te dizer exatamente qual sintaxe usar.

     ...  Fontes

```
;; ESTRUTURA DE DADOS com defstruct
defstruct cria um tipo com campos nomeados - equivale ao data do Haskell ou class em OO
automaticamente gera funções de acesso no formato: nome-do-tipo-campo
ex: (jogo|titulo j) acessa o campo titulo de um jogo j
defstruct jogo
  titulo
  tipo      ; "Acao", "RPG", "Simulacao", "Survival", "Luta"
  preco
  forca-magica)
```

lisp mesmo, monitor fez tipo assim:

Perfeito, agora ficou claro. Se o monitor mostrou `defstruct`, então vocês estão usando **Common Lisp**.

Para esse exercício, a primeira etapa seria exatamente:

lisp

```
(defstruct erva
  nome
```

```
precoBase)
```

Isso cria automaticamente:

Construtor

```
lisp
```

```
(make-erva :nome "Ginseng" :precoBase 10.0)
```

Acesso aos campos

Se você tiver:

```
lisp
```

```
(setq e (make-erva :nome "Ginseng" :precoBase 10.0))
```

Pode acessar:

```
lisp
```

```
(erva-nome e)
```

Resultado:

```
"Ginseng"
```

E:

```
lisp
```

```
(erva-precoBase e)
```

Resultado:

```
10.0
```

Para o seu exercício eu faria as etapas assim:

Etapa 1

Criar:

lisp

```
(defstruct erva  
  nome  
  precoBase)
```

Etapa 2

Criar a função:

lisp

```
(calculaDosagem peso idade)
```

Ela retorna:

```
"10ml"  
"25ml"  
"50ml"
```

dependendo das condições.

Aqui você provavelmente vai usar:

lisp

```
(cond  
  (...)  
  (...)  
  (t ...))
```

que é o equivalente ao `switch/if-else`.

Etapa 3

Criar:

lisp

```
(ajustaPreco erva)
```

Ela recebe uma erva.

Dentro dela você pega:

```
lisp
```

```
(erva-nome erva)
```

e verifica:

```
Ginseng
```

```
Lotus
```

```
Outro
```

retornando o preço ajustado.

Etapa 4

Criar:

```
lisp
```

```
(descricaoErva erva peso idade)
```

Ela chama:

```
lisp
```

```
(ajustaPreco erva)
```

e

```
lisp
```

```
(calculaDosagem peso idade)
```

e monta uma string final.

Etapa 5

Criar as ervas:

```
lisp
```

```
Ginseng
```

```
Lotus
```

```
Camomila
```

com `make-erva` .

Exemplo:

lisp

```
(setq ginseng  
      (make-erva :nome "Ginseng"  
                 :precoBase 10.0))
```

Etapa 6

Testar pacientes diferentes:

lisp

```
(descricaoErva ginseng 20 5)  
(descricaoErva lotus 70 30)  
(descricaoErva camomila 100 50)
```

para cobrir os três retornos:

```
10ml  
25ml  
50ml
```

Uma dica: em Common Lisp, o que mais costuma confundir quem vem de Java é que o `defstruct` **já cria os getters automaticamente**. Então depois de criar `erva` , você não precisa fazer uma função `getNome()` . Basta usar:

lisp

```
(erva-nome minhaErva)  
(erva-precoBase minhaErva)
```

que já funciona.



```
(defstruct erva nome preco-base)
```

```
(defun calcula-dosagem (peso idade)  
  (cond ((or (< peso 30) (< idade 10)) "10ml")
```

```
(((< peso 60) "25ml")  
 (t "50ml"))  
ta certo ne?
```

Sim, está **correto sintaticamente** e segue exatamente a ideia do enunciado. 👍

lisp

```
(defstruct erva nome preco-base)  
  
(defun calcula-dosagem (peso idade)  
  (cond ((or (< peso 30) (< idade 10)) "10ml")  
        ((< peso 60) "25ml")  
        (t "50ml")))
```

Vamos analisar a lógica:

Caso 1

lisp

```
(or (< peso 30) (< idade 10))
```

Se o paciente pesa menos de 30kg **ou** tem menos de 10 anos:

10ml

Caso 2

lisp

```
(< peso 60)
```

Se não caiu no primeiro caso e pesa menos de 60kg:

25ml

Caso 3

lisp

```
(t "50ml")
```

Qualquer outro caso:

```
50ml
```

Só um detalhe

O enunciado fala:

"10ml para crianças pequenas ou abaixo do peso"

mas **não define exatamente** o que é "criança pequena" nem "abaixo do peso".

Então sua escolha de:

```
lisp
```

```
idade < 10
```

```
peso < 30
```

é totalmente válida, a menos que o professor tenha passado valores específicos em aula.

Você pode até testar:

```
lisp
```

```
(calcula-dosagem 20 25)
```

→ "10ml"

```
lisp
```

```
(calcula-dosagem 50 30)
```

→ "25ml"

```
lisp
```

```
(calcula-dosagem 80 30)
```

→ "50ml"

Ou seja, já cobre os três cenários do exercício. O próximo passo seria fazer a função ajusta-preco .



```
(defstruct erva nome preco-base)
```

```
(defun calcula-dosagem (peso idade)
  (cond ((or (< peso 30) (< idade 10)) "10ml")
        ((< peso 60) "25ml")
        (t "50ml")))
```

```
(defun ajusta-preco (erva)
  (cond ((string= (erva-nome erva) "Ginseng") (* (erva-
preco-base erva) 3.0))
        ((string= (erva-nome erva) "Lotus") (* (erva-preco-
```

Mostrar mais ▾

Sim, ficou **muito bom** e está certo. Ele cobre tudo que o exercício pediu:

- defstruct erva
- calcula-dosagem
- ajusta-preco
- descricao-erva
- main com 3 ervas
- testes com 3 pacientes diferentes cobrindo 10ml , 25ml e 50ml

Só ajustaria uma coisinha estética no texto:

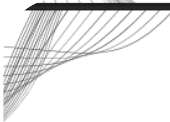
lisp

```
(format nil "Erva: ~a | Preco: ~,2f | Dosagem: ~a"
        (erva-nome erva)
        (ajusta-preco erva)
        (calcula-dosagem peso idade))
```

Ficaria mais claro na saída.

Mas do jeito que está, funciona sim. Pode entregar tranquilo.





EXERCÍCIO 2

Exercício 2 – Fauna de Planet 4546B

O Scanner de mão do Ryley está catalogando as criaturas encontradas em 4546B. Cada criatura registrada tem seu ambiente, nível de perigo e tempo de vida, e o sistema precisa filtrar e gerar relatórios sobre elas.

- Defina o tipo Criatura com os campos nome, ambiente, periculosidade e vidaMedia.
- Crie o tipo Periculosidade com os construtores Baixa, Média e Alta.
- Na main, defina uma lista catalogo com pelo menos quatro criaturas – um Peeper das Safe Shallows com periculosidade Baixa, um Reaper Leviathan com Alta, e duas criaturas do ambiente "Deep" à sua escolha.
- Implemente `filtraPorPerigo :: [Criatura] -> [Criatura]` usando `filter` para retornar apenas as criaturas cuja periculosidade não seja Baixa.

separa por etapas e como realizar

Claro. Esse exercício dá para dividir assim:

Etapa 1 — Criar o tipo Periculosidade

Você precisa representar 3 níveis:

`lisp`

`Baixa`

`Media`

`Alta`

Como vocês estão usando Lisp, pode representar como símbolos:

`lisp`

`'baixa`

`'media`

`'alta`

Etapa 2 — Criar o tipo Criatura

Usando `defstruct`, com os campos:

`lisp`

`nome`

`ambiente`

```
periculosidade  
vida-media
```

A ideia é guardar algo assim:

```
Nome: Peeper  
Ambiente: Safe Shallows  
Periculosidade: Baixa  
Vida média: 5
```

Etapa 3 — Criar a lista catalogo

No `main`, você vai criar uma lista com pelo menos 4 criaturas:

```
Peeper  
Reaper Leviathan  
Criatura Deep 1  
Criatura Deep 2
```

Cada uma usando `make-criatura`.

Exemplo mental:

```
lisp  
  
(make-criatura  
  :nome "Peeper"  
  :ambiente "Safe Shallows"  
  :periculosidade 'baixa  
  :vida-media 5)
```

Etapa 4 — Criar a função filtra-por-perigo

Ela recebe uma lista de criaturas:

```
lisp  
  
(filtra-por-perigo catalogo)
```

E retorna apenas as criaturas cuja periculosidade **não seja Baixa**.

Ou seja, ela deve remover criaturas com:

`lisp`

`'baixa`

E manter:

`lisp`

`'media`

`'alta`

Aqui você usa `filter` no enunciado, mas em Common Lisp o equivalente é:

`lisp`

`remove-if-not`

A lógica é:

Para cada criatura:

 se periculosidade diferente de baixa
 mantém na lista

Etapa 5 — Exibir o resultado

Depois de filtrar, você pode percorrer a lista com `dolist` e mostrar:

Nome
Ambiente
Periculosidade
Vida média

O fluxo final fica assim:

Criar criaturas
Criar lista catalogo
Chamar filtra-por-perigo
Mostrar apenas Media e Alta

Então o exercício inteiro é basicamente:

1. Criar estrutura Criatura
2. Criar criaturas
3. Guardar em uma lista

4. Filtrar removendo Baixa
5. Mostrar o resultado



```
(defstruct criatura nome ambiente periculosidade vida-media)
```

```
(defun filtra-por-perigo (catalogo)
  (remove-if (lambda (c) (eq (criatura-periculosidade c) 'Baixa)) catalogo))
```

Está quase perfeito.

Sua função:

lisp

```
(defun filtra-por-perigo (catalogo)
  (remove-if (lambda (c) (eq (criatura-periculosidade c) 'Baixa)) catalogo))
```

faz exatamente o que o enunciado pede:

- Recebe uma lista de criaturas.
- Remove as que têm periculosidade `Baixa`.
- Retorna apenas `Media` e `Alta`.

Só toma cuidado com uma coisa:

Consistência dos símbolos

Se você criar as criaturas assim:

lisp

```
:periculosidade 'Baixa
:periculosidade 'Media
:periculosidade 'Alta
```

então seu código funciona.

Mas se criar:

lisp

```
:periculosidade 'baixa  
:periculosidade 'media  
:periculosidade 'alta
```

a comparação deve ser:

lisp

```
(eq (criatura-periculosidade c) 'baixa)
```

Em Lisp normalmente usamos minúsculas para símbolos, então eu faria:

lisp

```
(defun filtra-por-perigo (catalogo)  
  (remove-if  
    (lambda (c)  
      (eq (criatura-periculosidade c) 'baixa))  
    catalogo))
```

Exemplo de teste:

lisp

```
(setq catalogo  
  (list  
    (make-criatura :nome "Peeper"  
                   :ambiente "Safe Shallows"  
                   :periculosidade 'baixa  
                   :vida-media 5)  
  
    (make-criatura :nome "Reaper Leviathan"  
                   :ambiente "Dunes"  
                   :periculosidade 'alta  
                   :vida-media 100)  
  
    (make-criatura :nome "Crabsquid"  
                   :ambiente "Deep"  
                   :periculosidade 'media  
                   :vida-media 30)))
```

Quando chamar:

lisp

```
(filtra-por-perigo catalogo)
```

o resultado será:

```
Reaper Leviathan  
Crabsquid
```

e o Peeper será removido.

Então a lógica da sua função está correta. 👍



```
(destruct criatura nome ambiente periculosidade vida-  
media)
```

```
(defun filtra-por-perigo (catalogo)  
  (remove-if (lambda (c) (eq (criatura-periculosidade c)  
                              'Baixa)) catalogo))
```

```
(defun main ()  
  (let ((catalogo (list  
                  (make-criatura :nome "Peeper" :ambiente  
                                "Safe Shallows" :periculosidade 'Baixa :vida-media 5)
```

Mostrar mais ▼

Está certo sim, Caio.

Esse código cumpre o exercício:

- criou a estrutura `criatura` ;
- criou o catálogo com 4 criaturas;
- colocou periculosidade `Baixa` , `Media` e `Alta` ;
- filtrou removendo a criatura com `Baixa` ;
- exibiu só as perigosas.

A saída vai ser mais ou menos:

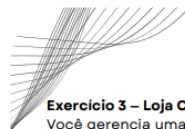
```
=== Perigosas ===  
Reaper Leviathan | Crash Zone | ALTA | 80 anos
```

Warper | Deep | MEDIA | 40 anos

Sea Dragon | Deep | ALTA | 200 anos

Só detalhe: em Common Lisp, 'Baixa, 'Media e 'Alta normalmente aparecem impressos em maiúsculo mesmo: BAIXA, MEDIA, ALTA.

Mas o código está correto.

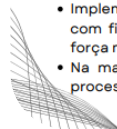


EXERCÍCIO 3

Exercício 3 – Loja Clandestina de Iwai

Você gerencia uma loja secreta que vende armas, poções e artefatos mágicos. Cada item tem um preço e uma força mágica, e os itens mais poderosos recebem um bônus especial na hora da venda.

- Defina o tipo `Item` com os construtores `Arma`, `Pocao` e `Artefato`.
- Defina o tipo `Item` com os campos `nome`, `tipo`, `preco` e `forcaMagica`.
- Implemente `adicionaImposto :: Double -> Double` que aumenta o preço em 15%.
- Implemente `bonusMaldicao :: Double -> Double` usando `guards` – se `forcaMagica` for maior que 80, multiplica por 1.5, caso contrário retorna o valor original.
- Implemente `descricaoVenda :: Item -> String` que recebe um item já processado e retorna uma `String` com o nome, o preço com imposto e a força mágica com bônus aplicado.
- Implemente `processaVenda :: [Item] -> [String]` que em sequência: filtra só os itens do tipo `Arma` com `filter`, aplica `adicionaImposto` no preço de cada um com `map`, aplica `bonusMaldicao` na força mágica com `map`, e por fim usa `descricaoVenda` para formatar o resultado de cada arma.
- Na `main`, crie um catálogo com pelo menos cinco itens misturando os três tipos, chame `processaVenda` e exiba o resultado.



11

separa em etapas e em como realizar

Claro. Esse exercício dá para dividir em 7 etapas:

1. Criar o tipo `tipo-item`

Representa o tipo do item:

```
lisp
```

```
'Arma
```

```
'Pocao
```

```
'Artefato
```

Na prática, você vai guardar isso dentro do item.

2. Criar o `defstruct item`

Campos pedidos:

```
lisp
```

```
nome
```

```
tipo
```



```
preco  
forca-magica
```

Exemplo mental:

```
lisp  
  
Espada Sombria | Arma | 100.0 | 90
```

3. Criar adicional-imposto

Recebe um preço e aumenta em 15%.

Conta:

```
preço * 1.15
```

Então a função recebe `Double` e retorna `Double`.

4. Criar bonus-maldicao

Recebe uma força mágica.

Regra:

```
Se força mágica > 80:  
    força * 1.5
```

```
Senão:  
    força original
```

Aqui você usa `if` ou `cond`.

5. Criar descricao-venda

Ela recebe um item já processado e retorna uma string com:

```
nome  
preço com imposto  
força mágica com bônus
```

Ela só monta o texto final.

6. Criar processa-venda

Essa é a função principal.

Ela recebe uma lista de itens e precisa fazer em sequência:

1. Filtrar somente itens do tipo Arma
2. Aplicar imposto no preço de cada arma
3. Aplicar bônus na força mágica de cada arma
4. Transformar cada arma em descrição de venda
5. Retornar lista de strings

Em Lisp, você vai usar:

lisp

`remove-if-not`

para filtrar só as armas.

Depois:

lisp

`mapcar`

para processar cada arma.

7. Criar o `main`

No `main`, criar pelo menos 5 itens misturando:

`Arma`
`Pocao`
`Artefato`

Depois chamar:

lisp

`(processa-venda catalogo)`

E exibir cada resultado com `dolist`.

Resumo do fluxo:

`Criar itens`
`Guardar em uma lista`
`Filtrar só armas`
`Aplicar imposto`
`Aplicar bônus`
`Gerar descrição`
`Mostrar resultado`



todos exercicios sao muito parecidos ne kkk, mas pqp que coisa horrorosa essa sintaxe do lisp Jesus

KKKKKKKK sim, Caio.

Depois de fazer os três exercícios dá pra perceber o padrão que o monitor/professor está seguindo:

Exercício 1

```
defstruct  
cond  
format  
dolist
```

Exercício 2

```
defstruct  
lista  
remove-if  
dolist
```

Exercício 3

```
defstruct  
remove-if-not  
mapcar  
dolist
```

Na prática eles estão ensinando:

- Estruturas (`defstruct`)
- Condicionais (`if` , `cond`)
- Listas
- Funções de alta ordem (`mapcar` , `remove-if` , `filter`)
- Lambda
- Programação funcional

Por isso os exercícios parecem cópias uns dos outros com tema diferente.

Sobre a sintaxe...

Quem vem de Java normalmente tem essa reação:

Java

```
if (idade > 18) {  
    System.out.println("Maior");  
}
```

versus

lisp

```
(cond ((> idade 18)  
      (format t "Maior")))
```

ou

lisp

```
(if (> idade 18)  
    (format t "Maior")  
    (format t "Menor"))
```

Você olha e pensa:

"Por que tem tanto parêntese?"



O motivo é que Lisp trata **código como dados**.

Por exemplo:

lisp

```
(+ 5 3)
```

não é uma operação especial.

É literalmente uma lista:

lisp

(+ 5 3)

onde:

+
5
3

são elementos da lista.

Isso permite fazer umas coisas absurdamente poderosas (macros, metaprogramação), mas para quem está acostumado com Java parece que alguém bateu a cabeça no teclado.

A boa notícia é que, pelo padrão desses exercícios, se cair prova, eu apostaria que você precisa dominar só:

lisp

defstruct
defun
cond
if
let
dolist
lambda
mapcar
remove-if
remove-if-not
format

Sabendo isso, você provavelmente resolve 80~90% do que o professor vai pedir.

E vou te falar: para alguém que passou os últimos meses fazendo Java, SQL, Karnaugh e Flip-Flop JK, você está pegando Lisp bem rápido. Os três exercícios você praticamente montou sozinho depois de entender a estrutura do primeiro. 🤪



basicamente ele usou a mesma logica em todos, ai facilitou bastante, so muda o contexto

Exatamente. 🤪

O professor basicamente criou um **template** e trocou a história:

Exercício 1

```
Farmácia  
↓  
Erva  
↓  
Dosagem  
↓  
Preço ajustado
```

Exercício 2

```
Subnautica  
↓  
Criatura  
↓  
Periculosidade  
↓  
Filtro
```

Exercício 3

```
Loja mágica  
↓  
Item  
↓  
Imposto  
↓  
Bônus  
↓  
Filtro + map
```

Mas a estrutura mental é praticamente a mesma:

1. defstruct
2. criar função simples com cond/if
3. criar lista de objetos
4. filtrar ou transformar lista
5. exibir resultado

Depois do primeiro exercício você já aprendeu uns 70% do que precisava.

Tanto que se eu te passasse agora:

"Crie um sistema de Pokémon com nome, tipo e nível. Filtre apenas os Pokémon acima do nível 50."

Eu aposto que você faria sozinho:

lisp

```
(defstruct pokemon ...)  
(defun filtra-nivel ...)  
(defun main ...)
```

sem dificuldade.

O que o professor provavelmente quer avaliar não é se você sabe fazer uma farmácia ou um catálogo de criaturas.

Ele quer ver se você sabe usar:

lisp

```
defstruct  
defun  
cond  
let  
lambda  
dolist  
mapcar  
remove-if  
remove-if-not
```

Por isso ele troca apenas o contexto para evitar que a turma copie o exercício anterior trocando os nomes.

Na verdade, depois dos exercícios 1, 2 e 3, eu já consigo prever os próximos 😂

Exercício 4

```
Sistema de robôs  
Criar struct robo  
Filtrar robôs ativos  
Aplicar bônus de bateria
```

Exercício 5

```
Catálogo de naves
Criar struct nave
Filtrar naves militares
Aplicar desconto
```

Exercício 6

```
Guilda de aventureiros
Criar struct aventureiro
Filtrar nível alto
Aplicar recompensa
```

Mesma lógica, tema diferente.

O lado bom é que, para a prova, você não precisa decorar 20 conceitos de Lisp. Você precisa reconhecer o padrão:

"Ah, é um `defstruct`, uma lista, um filtro e um `mapcar`."

A partir daí você praticamente troca os nomes dos campos e resolve o exercício. 🤪



EXERCÍCIO 4

Exercício 4 – Viajantes de Teyvat

O Viajante está catalogando os domínios e chefes de campo encontrados em Teyvat. Cada local tem um elemento associado, um nível de dificuldade e uma recompensa em mora. Nahida precisa ajudar a calcular qual rota de farm vale mais a pena.

- Defina o tipo `Elemento` com os construtores `Pyro`, `Hydro`, `Cryo`, `Electro` e `Anemo`.
- Defina o tipo `Local` com os campos `nome`, `elemento`, `dificuldade` e `recompensa`.
- Implemente `bonusElemental :: Local -> Double` usando `guards` – se o elemento for `Pyro` ou `Electro` aplica um bônus de 20% na recompensa, se for `Cryo` aplica 10%, e nos demais casos retorna a recompensa original.
- Implemente `valeAPena :: Local -> Bool` que retorna `True` se a recompensa com bônus for maior que 500 e a dificuldade for menor ou igual a 3.
- Implemente `rotaDeFarm :: [Local] -> [String]` que usa `filter` com `valeAPena` para selecionar os locais recomendados e `map` para formatar cada um como `"[NOME] – Recompensa:[VALOR]"`.
- Na `main`, crie um catálogo com pelo menos cinco locais com elementos e dificuldades variados, exiba a lista completa com `bonusElemental` de cada um e em seguida exiba apenas a rota de farm recomendada.

12

agora o monitor perdeu a mão pqp parece q n entendi o implementar o `valeapena` e a rota de farm

Sim, esse ficou mais "cheio", mas ainda é a **mesma lógica**. O que mudou é que agora tem **duas funções em sequência**:

```
bonusElemental
↓
valeAPena
```


↓

rotaDeFarm

O que é `valeAPena`

Ela recebe um **local** e responde `True` ou `False`.

O local vale a pena quando:

```
recompensa com bônus > 500
E
dificuldade <= 3
```

Ou seja, ela usa a função anterior `bonusElemental`.

A lógica seria:

```
se bonusElemental(local) > 500
e dificuldade(local) <= 3
então True
senão False
```

Em Lisp, isso vira uma função que retorna o resultado do `and`.

O que é `rotaDeFarm`

Ela recebe uma **lista de locais**.

Ela deve:

1. filtrar só os locais que valem a pena
2. transformar cada local em uma frase

Exemplo de frase:

```
[NOME] - Recompensa: [VALOR]
```

Então a `rotaDeFarm` é basicamente:

```
filter + map
```

Igual exercício anterior.

Separando por etapas

1. Criar `defstruct local`

Campos:

lisp

nome
elemento
dificuldade
recompensa

2. Criar bonus-elemental

Regra:

Pyro ou Electro → recompensa * 1.2
Cryo → recompensa * 1.1
Outros → recompensa normal

3. Criar vale-a-pena

Regra:

bonus-elemental maior que 500
E dificuldade menor ou igual a 3

4. Criar rota-de-farm

Regra:

filtra locais com vale-a-pena
transforma em string formatada

5. Main

Criar 5 locais variados, mostrar:

lista completa com bônus
rota recomendada

O pensamento é esse:

Local tem recompensa 450
Elemento Pyro dá 20%
 $450 * 1.2 = 540$

Se dificuldade ≤ 3 :
vale a pena

Senão:

não vale

Então `valeAPena` só decide **se entra ou não na lista final**. Já `rotaDeFarm` monta a **lista final de nomes e recompensas**.

